



Реализация REST API на основе HAL контракта

Цель работы

На лекции мы познакомились с основами HATEOAS. Сегодня мы сделаем следующий шаг и полностью интегрируем этот принцип в наш API, охватив не только одиночные ресурсы, но и коллекции, а также пагинацию. Мы превратим наш API из простого поставщика данных в интерактивную систему, которая направляет клиента.

Вводные

Представьте себе сложный веб-интерфейс (например, админ-панель), который работает с нашим API.

Без HATEOAS

Чтобы отобразить таблицу книг с пагинацией, фронтенд-разработчик должен вручную конструировать URL для каждой страницы - ?page=0, ?page=1 и т.д. Если логика пагинации изменится (например, параметры переименуют в r и s), фронтенд сломается.

С HATEOAS

API в ответе на первую страницу сразу отдает готовые ссылки:

"next": "http://.../api/books?page=1&size=10",

"last": "http://.../api/books?page=5&size=10".

Фронтенду нужно лишь отобразить кнопки "Вперед" и "Назад" и привязать к ним эти готовые URL. Система становится устойчивой к изменениям.

Это позволяет создавать сложные, слабосвязанные системы, где клиент и сервер могут развиваться независимо друг от друга.

Нам нужно:

Проект с контрактом (у нас **books-api-contract**), созданный на предыдущем занятии.

Откройте у него файл pom.xml, там внесите зависимость с поддержкой HATEOAS

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-  
hateoas</artifactId>  
</dependency>
```

Обновите Maven-проект, чтобы изменения вступили в силу



СОП

HATEOAS (Hypermedia as the Engine of Application State) - это ограничение архитектуры REST, которое гласит: клиент должен взаимодействовать с приложением полностью через гипермедиа, предоставляемые сервером в ответах. Цель - сделать API самообнаруживаемым (self-discoverable). Вместо того чтобы клиент заранее знал все возможные эндпоинты, сервер сам подсказывает ему, какие действия доступны из текущего состояния.

Что там есть?

Класс RepresentationModel

Базовый класс в Spring HATEOAS для любого объекта, который должен содержать ссылки. Наши DTO (например, BookResponse) будут наследоваться от этого класса, чтобы получить возможность хранить и отображать коллекцию ссылок.

Класс EntityModel<T>

Универсальный контейнер (generic wrapper) для оборачивания любого доменного объекта или DTO с целью добавления к нему ссылок. Позволяет добавить ссылки к любому объекту, не изменяя его исходный класс. Часто используется как возвращаемый тип из контроллера.

Класс Link

Объект, представляющий одну гипермедийную ссылку. Состоит из URL (href) и отношения. Является строительным блоком для HATEOAS. Отношение rel объясняет, что означает эта ссылка. Стандартные отношения — self (ссылка на сам ресурс), next, previous, но вы можете создавать и свои (например, author, books).

Паттерн RepresentationModelAssembler

Интерфейс-конвертер, который преобразует доменный объект (или DTO) в RepresentationModel (чаще всего в EntityModel). Это лучшая практика для инкапсуляции логики создания ссылок. Вместо того чтобы «засорять» контроллер кодом для построения URL, мы выносим эту логику в отдельный класс-компонент (ассемблер).

Класс WebMvcLinkBuilder

Основной строитель ссылок в Spring HATEOAS. Позволяет создавать объекты Link типобезопасным способом, ссылаясь напрямую на методы контроллера. Это защищает от ошибок при рефакторинге URL.

Основные команды:



СОП

- `linkTo(methodOn(Controller.class).method(args...))`
 - Создает заготовку ссылки, указывая на конкретный метод контроллера
- `.withSelfRel()`
 - Добавляет к ссылке отношение `rel="self"`.
- `.withRel("relation-name")`
 - Добавляет к ссылке кастомное отношение, например `rel="author"`.

Класс `CollectionModel<T>`

Универсальный контейнер для представления коллекции ресурсов (например, списка авторов). Позволяет обернуть `List<EntityModel<T>>` и добавить ссылки, относящиеся ко всей коллекции. Например, ссылка `self` на саму коллекцию (`/api/authors`).

Класс `PagedModel<T>`

Специализированный `CollectionModel` для пагинированных данных. Помимо содержимого страницы и ссылок на отдельные элементы, он содержит метаданные о пагинации (`size`, `totalElements`, `totalPages`, `number`) и навигационные ссылки (`first`, `prev`, `self`, `next`, `last`).

Инструмент `PagedResourcesAssembler<T>`

Вспомогательный компонент Spring HATEOAS, который значительно упрощает преобразование объекта `Page<T>` (стандартный объект пагинации из Spring Data) в `PagedModel<EntityModel<T>>`. Автоматизирует всю рутинную работу по созданию навигационных ссылок для пагинации. Нам не нужно вручную вычислять URL для следующей или предыдущей страницы.

Практический разбор

В начале мы переработаем контракт, соблюдая практику `Contract First`.

Контракт - лицо нашего API. Он должен четко отражать, что мы возвращаем гипермедийные ответы.

Обновляем `pom.xml`

Убедитесь, что зависимость `spring-boot-starter-hateoas` добавлена, как описано выше, надеюсь вы внимательно читаете инструкции 😊

Модифицируем DTO

Наши `AuthorResponse` и `BookResponse` должны иметь возможность содержать ссылки (при этом уходим от `record` для наследования):



```
@Relation(collectionRelation = "authors", itemRelation = "author")
public class AuthorResponse extends RepresentationModel<AuthorResponse> {

    private final Long id;
    private final String firstName;
    private final String lastName;

    public AuthorResponse(Long id, String firstName, String lastName) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public Long getId() {
        return id;
    }

    public String getFirstName() {
        return firstName;
    }

    public String getLastName() {
        return lastName;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        if (!super.equals(o)) return false;
        AuthorResponse that = (AuthorResponse) o;
        return Objects.equals(id, that.id) && Objects.equals(firstName,
that.firstName) && Objects.equals(lastName, that.lastName);
    }

    @Override
    public int hashCode() {
        return Objects.hash(super.hashCode(), id, firstName, lastName);
    }
}

@Relation(collectionRelation = "books", itemRelation = "book")
public class BookResponse extends RepresentationModel<BookResponse> {

    private final Long id;
    private final String title;
    private final String isbn;
    private final AuthorResponse author;
    private final LocalDateTime createdAt;

    public BookResponse(Long id, String title, String isbn, AuthorResponse
author, LocalDateTime createdAt) {
        this.id = id;
        this.title = title;
        this.isbn = isbn;
        this.author = author;
        this.createdAt = createdAt;
    }
}
```



СОП

```
public Long getId() {
    return id;
}

public String getTitle() {
    return title;
}

public String getIsbn() {
    return isbn;
}

public AuthorResponse getAuthor() {
    return author;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    if (!super.equals(o)) return false;
    BookResponse that = (BookResponse) o;
    return Objects.equals(id, that.id) && Objects.equals(title,
that.title) && Objects.equals(isbn, that.isbn) && Objects.equals(author,
that.author) && Objects.equals(createdAt, that.createdAt);
}

@Override
public int hashCode() {
    return Objects.hash(super.hashCode(), id, title, isbn, author,
createdAt);
}
}
```

Если вы используете LOMБОК

```
@Value // Неизменяемый класс, аналог record
@EqualsAndHashCode(callSuper = false) // Важно для HATEOAS
@Relation(collectionRelation = "authors", itemRelation = "author")
public class AuthorResponse extends RepresentationModel<AuthorResponse> {
    Long id;
    String firstName;
    String lastName;
}
```

Контракт готов. Он явно декларирует, что API общается на языке гипермедиа.

Пересобираем JAR, обновляем его в проекте реализации, копируя в директорию lib, а затем меняем версию на новую (да цифру просто увеличиваем, обновляем maven)

Работаем в проекте реализации



Обновляем POM, добавляем:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Теперь создадим компоненты, которые будут инкапсулировать логику построения
ссылки

Создаем в проекте пакет `assemblers`, в нем будут классы:

AuthorModelAssembler

```
@Component
public class AuthorModelAssembler implements
RepresentationModelAssembler<AuthorResponse, EntityModel<AuthorResponse>> {

    @Override
    public EntityModel<AuthorResponse> toModel(AuthorResponse author) {
        return EntityModel.of(author,

linkTo(methodOn(AuthorController.class).getAuthorById(author.getId())).withSelfRel(),

linkTo(methodOn(BookController.class).getAllBooks(author.getId(), 0,
10)).withRel("books"),

linkTo(methodOn(AuthorController.class).getAllAuthors()).withRel("collection"
)
        );
    }

    @Override
    public CollectionModel<EntityModel<AuthorResponse>>
toCollectionModel(Iterable<? extends AuthorResponse> entities) {
        return RepresentationModelAssembler.super.toCollectionModel(entities)

.add(linkTo(methodOn(AuthorController.class).getAllAuthors()).withSelfRel());
    }
}
```

BookModelAssembler

```
@Component
public class BookModelAssembler implements
RepresentationModelAssembler<BookResponse, EntityModel<BookResponse>> {

    @Override
    public EntityModel<BookResponse> toModel(BookResponse book) {
        return EntityModel.of(book,

linkTo(methodOn(BookController.class).getBookById(book.getId())).withSelfRel()
),
}
```



```
linkTo(methodOn(AuthorController.class).getAuthorById(book.getAuthor().getId(
))) .withRel("author"),
        linkTo(methodOn(BookController.class).getAllBooks(null, 0,
10)) .withRel("collection")
    );
}
```

Обновляем контроллеры

```
@RestController
public class BookController implements BookApi {

    private final BookService bookService;
    private final BookModelAssembler bookModelAssembler;
    private final PagedResourcesAssembler<BookResponse>
pagedResourcesAssembler;

    public BookController(BookService bookService, BookModelAssembler
bookModelAssembler, PagedResourcesAssembler<BookResponse>
pagedResourcesAssembler) {
        this.bookService = bookService;
        this.bookModelAssembler = bookModelAssembler;
        this.pagedResourcesAssembler = pagedResourcesAssembler;
    }

    @Override
    public EntityModel<BookResponse> getBookById(Long id) {
        BookResponse book = bookService.findBookById(id);
        return bookModelAssembler.toModel(book);
    }

    @Override
    public PagedModel<EntityModel<BookResponse>> getAllBooks(Long authorId,
int page, int size) {
        // Наш сервис возвращает кастомный PagedResponse, а
PagedResourcesAssembler ожидает Page из Spring Data, а у нас In Memory
реализация.
        // Поэтому мы делаем простую адаптацию.
        PagedResponse<BookResponse> pagedResponse =
bookService.findAllBooks(authorId, page, size);
        Page<BookResponse> bookPage = new PageImpl<>(
            pagedResponse.content(),
            PageRequest.of(pagedResponse.pageNumber(),
pagedResponse.pageSize()),
            pagedResponse.totalElements()
        );

        // PagedResourcesAssembler автоматически создаст PagedModel со всеми
ссылками пагинации
        return pagedResourcesAssembler.toModel(bookPage, bookModelAssembler);
    }

    @Override
    public ResponseEntity<EntityModel<BookResponse>> createBook(BookRequest
request) {
        BookResponse createdBook = bookService.createBook(request);
        EntityModel<BookResponse> entityModel =
bookModelAssembler.toModel(createdBook);

        return ResponseEntity
```



СОН

```
.created(entityModel.getRequiredLink("self").toUri())
.body(entityModel);
}

@Override
public EntityModel<BookResponse> updateBook(Long id, UpdateBookRequest
request) {
    BookResponse updatedBook = bookService.updateBook(id, request);
    return bookModelAssembler.toModel(updatedBook);
}

@Override
public void deleteBook(Long id) {
    bookService.deleteBook(id);
}
}

@RestController
public class AuthorController implements AuthorApi {

    private final AuthorService authorService;
    private final AuthorModelAssembler authorModelAssembler;

    public AuthorController(AuthorService authorService, AuthorModelAssembler
authorModelAssembler) {
        this.authorService = authorService;
        this.authorModelAssembler = authorModelAssembler;
    }

    @Override
    public CollectionModel<EntityModel<AuthorResponse>> getAllAuthors() {
        List<AuthorResponse> authors = authorService.findAll();
        return authorModelAssembler.toCollectionModel(authors);
    }

    @Override
    public EntityModel<AuthorResponse> getAuthorById(Long id) {
        AuthorResponse author = authorService.findById(id);
        return authorModelAssembler.toModel(author);
    }

    @Override
    public ResponseEntity<EntityModel<AuthorResponse>>
createAuthor(AuthorRequest request) {
        AuthorResponse createdAuthor = authorService.create(request);
        EntityModel<AuthorResponse> entityModel =
authorModelAssembler.toModel(createdAuthor);

        return ResponseEntity
            .created(entityModel.getRequiredLink("self").toUri())
            .body(entityModel);
    }

    @Override
    public EntityModel<AuthorResponse> updateAuthor(Long id, AuthorRequest
request) {
        AuthorResponse updatedAuthor = authorService.update(id, request);
```



СОП

```
        return authorModelAssembler.toModel(updatedAuthor);
    }

    @Override
    public void deleteAuthor(Long id) {
        authService.delete(id);
    }
}
```

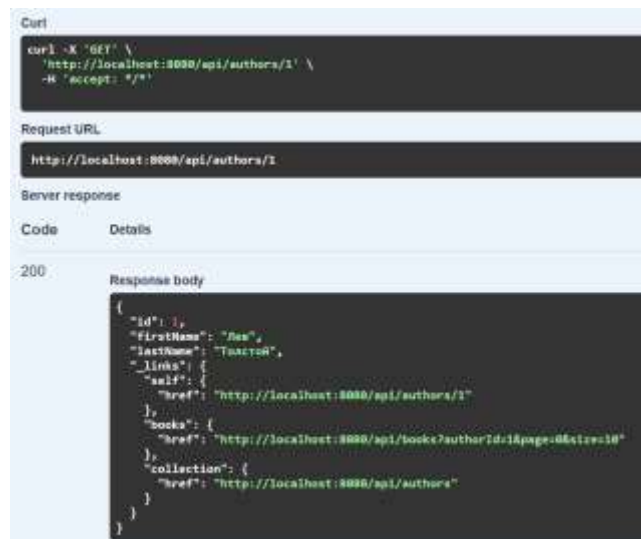
Обновляем сервисы (чиним геттеры и сеттеры, так как там теперь классы)

Остается только main (подключили зависимость от Spring Data, отключаем автоконфигурацию чтобы не просил БД), + включаем HAL

```
@SpringBootApplication(
    scanBasePackages = {"edu.rutmiit.demo.demorest",
        "edu.rutmiit.demo.booksapicontract"},
    exclude = {DataSourceAutoConfiguration.class}
)
@EnableHypermediaSupport(type = EnableHypermediaSupport.HypermediaType.HAL)
public class DemoRestApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoRestApplication.class, args);
    }
}
```

Проверяем, все ок:



Задания для самостоятельной Работы

Обновите контракт своего сервиса и на основе примера реализуйте новый контракт.